DA3408 Assignment 3 (K Kevin Sona DA24B007)

1 Cluster setup

Both clusters run as Docker Compose services on one laptop (16 cores, 14 GB RAM). The Spark cluster reuses the standalone cluster image from the class repository, with one master and two workers, and jobs are started with spark-submit from the master container. The Ray cluster uses the official Ray 2.59 image, with one head node and two worker nodes, and the script joins the running cluster with `ray.init(address="auto")`. In both cases the coordinator does no data processing (the Ray head advertises zero CPUs) and the data folder is mounted into every container as a shared directory. I gave both sets of workers exactly the same hard limits so that the comparison is fair.

Table 1: Cluster configuration.

	Spark 3.3.4	Ray 2.59
Coordinator	1 master, which also runs the driver	1 head with 0 CPUs, which also runs the driver
Workers	2 workers, 2 cores each	2 workers, 2 CPUs each
Hard limit per worker	2 CPUs and 3 GB memory	2 CPUs and 3 GB memory
How the 3 GB is used	2 GB Java heap, rest for Python UDF processes	0.5 GB object store, rest for Python workers
Web UI	Master UI on port 9090, application UI on 4040	Dashboard on port 8265

2 Pipeline and parity

To check parity I compared the two outputs with `compare_outputs.py`, which reads them in batches and compares the schema, the row count, the sum of every numeric column, an order independent fingerprint of every row, and the 24 hourly averages. Every check matched (Table 2).

Table 2: Parity check between the Spark and Ray outputs.

Check	Spark	Ray	Result
Rows after cleansing	17,870,679	17,870,679	Match
Rows in the final output	17,865,181	17,865,181	Match
Sum of trip_distance	60,134,887.92	60,134,887.92	Match
Sum of total_amount	506,959,678.49	506,959,678.49	Match
Sum of speed_mph (the UDF output)	197,088,783.64	197,088,783.64	Match
Fingerprint of all rows	14672325616244656533	14672325616244656533	Match
Largest difference in the 24 hourly averages	0.000000		Match

3 Execution time

I ran each pipeline three times on all six months and report the median. Each script records the wall clock time of every stage and writes it to a results file, so none of the numbers below were typed in by hand. Spark took between 127 and 160 seconds and Ray took between 1,044 and 1,188 seconds. The first Spark run was the slowest because the files were being read from disk for the first time.

Table 3: Median wall clock time per stage in seconds (three runs each).

Stage	Spark	Ray	Ray / Spark
Ingestion	6.6	5.9	0.9
Cleansing	24.5	42.4	1.7
Transformation (joins, UDF, hourly feature)	83.2	751.4	9.0
Export	22.4	301.2	13.5
Total	143.3	1,085.1	7.6

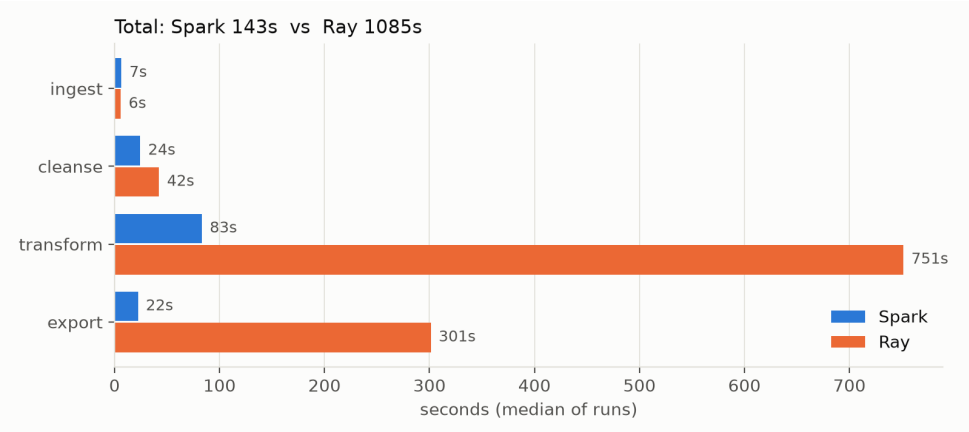


Figure 1: Median time per stage. Almost all of Ray's extra time is in the joins and in the export, which repeats the final join.

4 Resource utilisation

I sampled CPU and memory of every container with docker stats about every two seconds during each run. CPU is reported per core, so a worker with two cores peaks at 200 percent.

Table 4: Resource use of the workers and coordinator (median over the three runs).

	Spark workers	Ray workers	Spark master	Ray head
Peak CPU	207%	211%	260%	92%
Average CPU during the run	174%	57%	17%	26%
Peak memory (limit 3 GB)	2.66 GB	2.82 GB	1.11 GB	1.40 GB

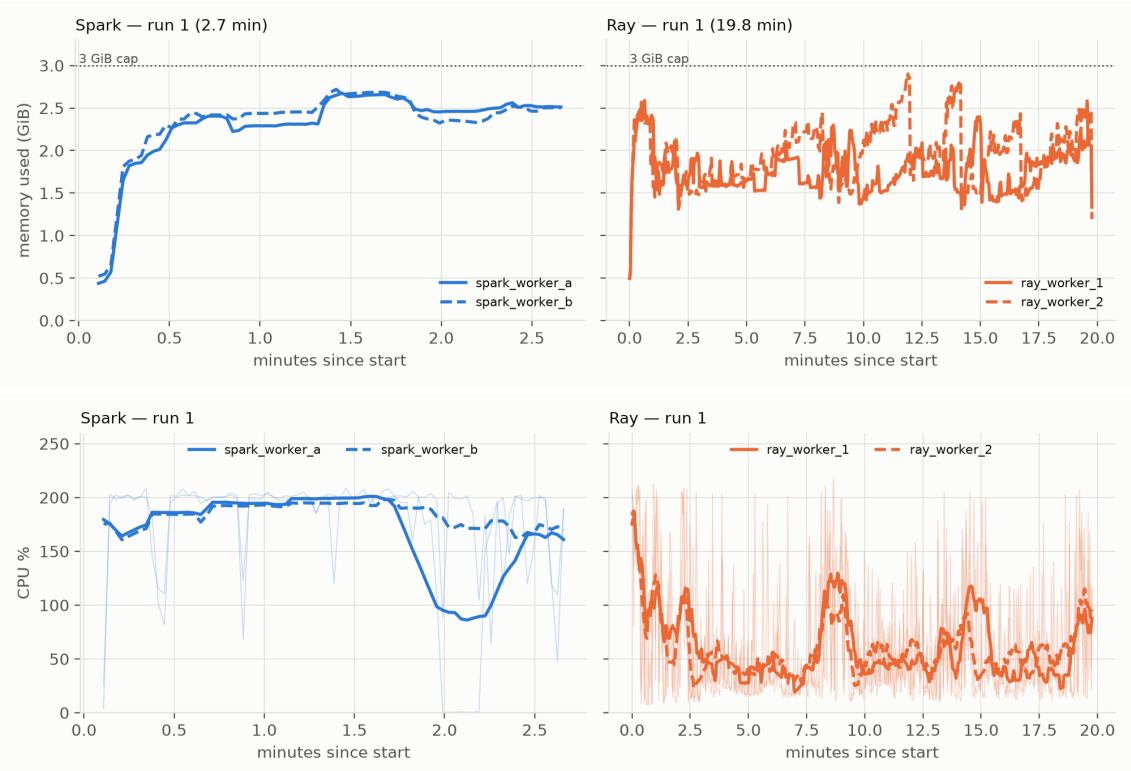


Figure 2: Worker memory (top) and CPU (bottom, smoothed over 30 seconds) during the first run. Spark's memory stays flat around 2.5 GB and both cores stay busy. Ray's memory swings close to the 3 GB limit and its CPU mostly sits between 25 and 60 percent because it is waiting for data to be written to and read from disk.

5 Python UDF deep dive

To isolate the cost of the custom Python function, each script runs the same speed rule two ways on the cleaned data, after the main pipeline. The native version uses the engine's own fast path, which is a Spark SQL expression inside the Java engine for Spark and a vectorised pandas expression for Ray. The Python version is the exact same row by row function, `trip_speed_mph`, that the pipeline uses. Each version is run twice and the faster time is kept. Both engines produced the same sum of 197,088,783.64 for the Python version.

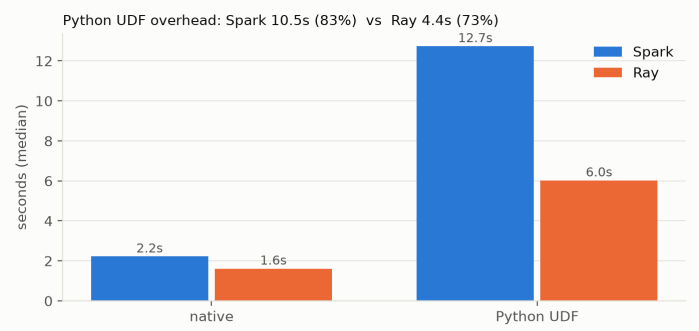


Figure 3: Median time to compute the speed for 17.9 million rows.

Table 5: UDF timing (median of three runs, seconds).

	Native	Python	Overhead	Share
Spark	2.2	12.7	10.5	83%
Ray	1.6	6.0	4.4	73%

In Spark the data lives in the Java engine. To run a Python function, every executor starts separate Python processes, and each row has to be converted, sent to Python, processed and sent back. The Spark UI shows this step as a node

called BatchEvalPython. That round trip is 10.5 of the 12.7 seconds, or 83 percent of the time. In Ray the workers are Python processes and the data is already in Arrow and pandas form, so the same function runs directly on the data with no translation. Ray's Python version was therefore about twice as fast as Spark's. The remaining 4.4 seconds of overhead in Ray is simply the cost of a Python loop compared with vectorised code, which both engines would pay. This is the Python native advantage of Ray: when the work is custom Python, Ray removes the cost of crossing between two languages. On the Spark side, a pandas UDF that sends data in Arrow batches would narrow the gap, but it would still need the crossing.

6 Discussion, limitations and conclusion

Some limits should be kept in mind. Both clusters run on one laptop, so the two workers share one SSD and the network between them is virtual. On a real cluster the shuffles would be slower in absolute terms, but they would affect both engines in the same way. Ray needed tuning to fit in 3 GB, and it would likely run much faster with more memory per worker, so the gap in total time is partly a result of the tight memory limit. One Ray run failed because the laptop disk passed 95 percent while Ray was spilling, and one Spark attempt was started while an earlier run still held all cores, so it never ran. Both were replaced by complete runs and are not included in the medians.

For a BI first project, where the work is mainly large joins, group-bys and reports over structured tables, I would choose Spark. It was 7.6 times faster here, it kept its CPUs busy, and its shuffle handled data larger than memory without any tuning. For an AI first project, where the work is mainly custom Python such as feature functions, model inference or calls into Python libraries, I would choose Ray. It ran the same Python function twice as fast, it needs no Java runtime, and it fits naturally with the rest of a Python machine learning stack. For this assignment's pipeline, which is dominated by heavy joins, the overall winner is Spark, while Ray wins the Python UDF comparison.

7 Performance tuning note (use of AI)

I used an AI assistant (Claude) to help translate the pipeline into Ray Data and to diagnose performance problems. I set up the cluster configuration and networking myself. The changes it suggested, and what they did, are listed here. In Spark, it suggested filling the cache of the joined table with a count before reusing it, because two parallel branches were recomputing the same joins and UDF; on one month of data this cut the transform time from 34 to 28 seconds and made shuffle reads equal shuffle writes. It also suggested warming the cache before the UDF timing so that the native baseline was measured fairly. In Ray, it suggested replacing a twelve column group-by used for removing duplicates, which took 233 seconds for one month, with a group by pickup zone followed by pandas drop_duplicates, which took 2.7 seconds and gives the same result. To stop the Ray job from deadlocking and running out of memory in 3 GB, it suggested the sort based shuffle for group-bys, an explicit memory reservation for the join helpers, 32 join partitions, smaller blocks, running the two joins one after the other, and a smaller object store, which I applied in the cluster configuration. It also caught a type mismatch where the shuffle widened the zone IDs from 32 to 64 bit integers, which made the joins fail. All benchmark numbers in this report come from real runs on the clusters described above.

Appendix: UI screenshots

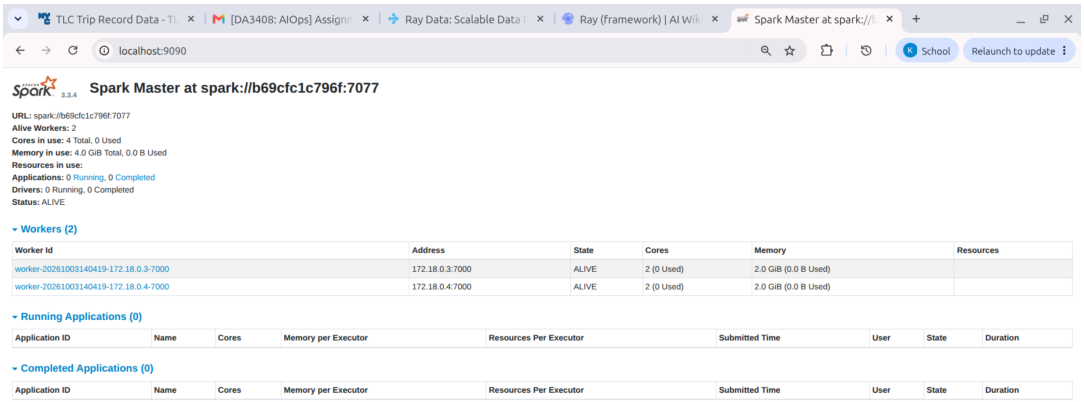


Figure 4: Spark Master UI (port 9090): two alive workers with 2 cores and 2 GB each.

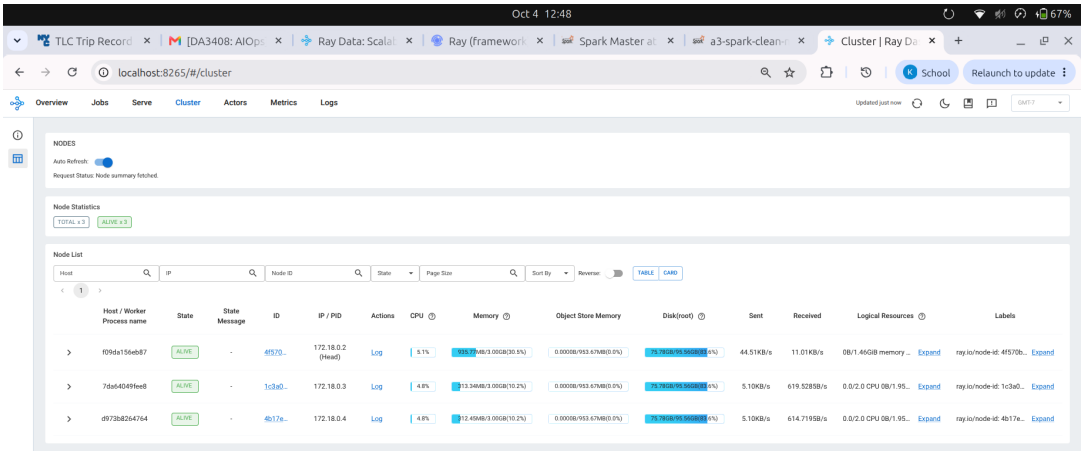


Figure 5: Ray Dashboard cluster view (port 8265): the head node and two worker nodes.

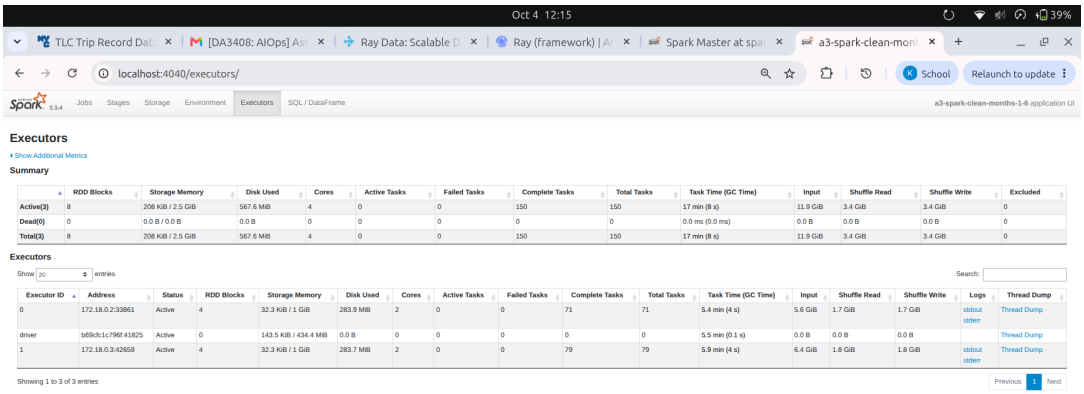


Figure 6: Spark application UI (port 4040), Executors tab of a 6 month run: two executors on the two workers, with 71 and 79 tasks.

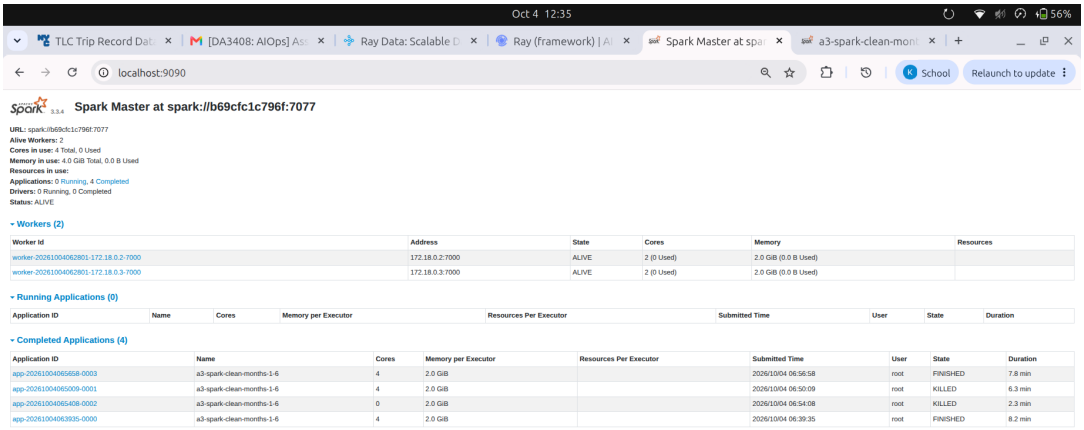


Figure 7: Spark Master UI after the benchmark. Runs 0000, 0001 and 0003 completed and saved their metrics (0001 was stopped later while holding its UI open). Run 0002 received no cores and was discarded.

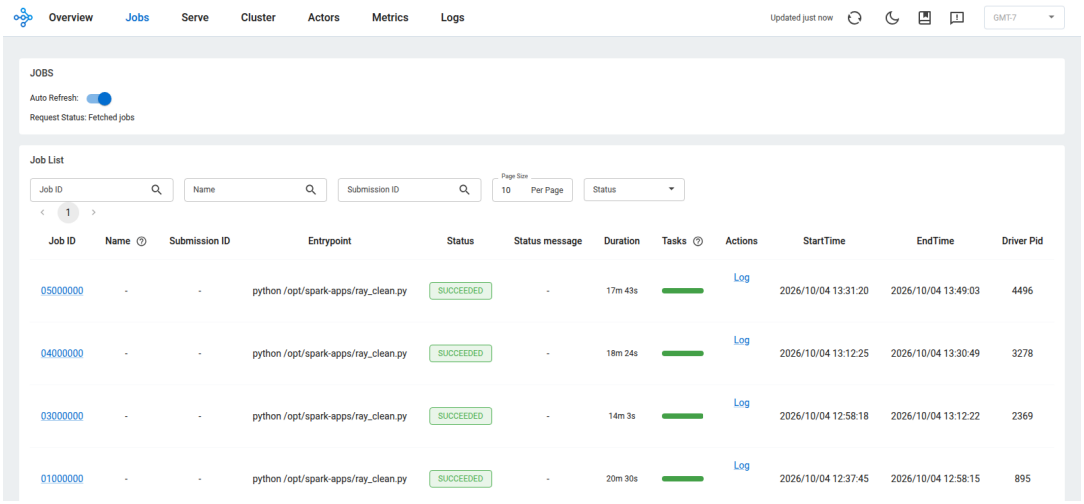


Figure 8: Ray Dashboard jobs page. Jobs 01, 04 and 05 are the three benchmark runs. Job 03 stopped with a disk full error and is not used.